

Appendix B

320Caml Specification

The following is a specification for a subset of OCaml used during the first part of the course.

B.1 Literals

Syntax

```
<int> ::= 0 | 1 | 2 | ...  
<float> ::= 0. | 1. | 2. | ...  
<bool> ::= true | false  
<expr> ::= <int> | <float> | <bool>  
<ty> ::= int | float | bool
```

Literals are well-formed expressions by *fiat*. In the case of Boolean literals, we can make the syntax explicit: there are two literals, `true` and `false`. Infinite sets of literals (e.g., in the case of integers and floating-point numbers) are defined using regular expressions and are dealt with during *lexical analysis*. We additionally state that `int`, `float`, and `bool` are well-formed types.

Example B.1.1. The integer literal `-2` and the Boolean literal `true` are well-formed expressions.

Typing

$\frac{\text{n is an integer literal}}{\Gamma \vdash n : \text{int}} \text{INTLIT}$	$\frac{\text{n is an float literal}}{\Gamma \vdash n : \text{float}} \text{FLOATLIT}$	$\frac{}{\Gamma \vdash \text{true} : \text{bool}} \text{TRUE}$
$\frac{}{\Gamma \vdash \text{false} : \text{bool}} \text{FALSE}$		

The rules for literals are *axioms*, i.e., they have no premises. When building a derivation, these rules will appear as leaves of the derivation tree.

- ▷ INTLIT says every integer literal is of type `int` in any context.
- ▷ FLOATLIT says every floating-point literal is of type `float` in any context.

- ▷ TRUE says **true** is of type **bool** in any context.
- ▷ FALSE says **false** is of type **bool** in any context.

Example B.1.2. The integer literal **35** is of type **int** in the empty context, i.e., the following is a valid derivation.

$$\frac{}{\emptyset \vdash 35 : \text{int}} \text{INTLIT}$$

Semantics

$$\begin{array}{c} \frac{\text{n is an integer literal for } n}{n \Downarrow n} \text{INTLITE} \qquad \frac{\text{n is an float literal for } n}{n \Downarrow n} \text{FLOATLITE} \qquad \frac{}{\text{true} \Downarrow \top} \text{TRUEE} \\ \frac{}{\text{false} \Downarrow \perp} \text{FALSEE} \end{array}$$

- ▷ INTLITE says an integer literal evaluates to the number it denotes.
- ▷ FLOATLITE says a floating-point literal evaluates to the number it denotes.
- ▷ TRUEE says the literal **true** evaluates to the Boolean value $\top$.
- ▷ FALSEE says the literal **false** evaluates to the Boolean value $\perp$.

Example B.1.3. The floating-point literal **3.14** evaluates to the number 3.14, i.e., the following is a valid derivation.

$$\frac{}{3.14 \Downarrow 3.14} \text{FLOATLITE}$$

B.2 Variables

Syntax

$$\begin{array}{l} \langle \text{var} \rangle ::= x \mid \text{foo_bar} \mid \text{abc}' \mid \dots \\ \langle \text{expr} \rangle ::= \langle \text{var} \rangle \end{array}$$

Variables is a well-formed expression. As with number literals, we can't make the syntax of a variable explicit, but we'll take a variable to be alphanumeric with underscores and single-quotes, not starting with a capital letter.

Typing

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau} \text{VAR}$$

VAR says if the variable x is declared to be of type τ in the context Γ , then x is of type τ in the context Γ .

Example B.2.1. Since $(x : \text{int})$ appears in the context $\{x : \text{int}, y : \text{bool}\}$, the variables x is of type int in this context, i.e., the following is a valid derivation.

$$\frac{}{\{x : \text{int}, y : \text{bool}\} \vdash x : \text{int}} \text{VAR}$$

Semantics

There are no semantic rules for variables. If evaluating an expression requires evaluating a variable, then the expression is not well-scoped.

B.3 Operators

Syntax

```

<op> ::= + | - | * | / | mod
      | +. | -. | *. | /.
      | && | ||
      | = | <> | < | <= | > | >=
<expr> ::= <expr> <op> <expr>

```

The first rule says the symbols $+$, $-$, $*$, $/$, mod , etc. are well-formed operators. The second says that if $\square$ is a well-formed operator and e_1 and e_2 are well-formed expressions, then $e_1 \square e_2$ is a well-formed expression.

Typing

$$\begin{array}{c}
\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}} \text{ ADDINT} \qquad \frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 - e_2 : \text{int}} \text{ SUBINT} \\
\\
\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 * e_2 : \text{int}} \text{ MULINT} \qquad \frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 / e_2 : \text{int}} \text{ DIVINT} \\
\\
\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 \text{ mod } e_2 : \text{int}} \text{ MODINT} \qquad \frac{\Gamma \vdash e_1 : \text{float} \quad \Gamma \vdash e_2 : \text{float}}{\Gamma \vdash e_1 +. e_2 : \text{float}} \text{ ADDFLOAT} \\
\\
\frac{\Gamma \vdash e_1 : \text{float} \quad \Gamma \vdash e_2 : \text{float}}{\Gamma \vdash e_1 -. e_2 : \text{float}} \text{ SUBFLOAT} \qquad \frac{\Gamma \vdash e_1 : \text{float} \quad \Gamma \vdash e_2 : \text{float}}{\Gamma \vdash e_1 *. e_2 : \text{float}} \text{ MULFLOAT} \\
\\
\frac{\Gamma \vdash e_1 : \text{float} \quad \Gamma \vdash e_2 : \text{float}}{\Gamma \vdash e_1 /. e_2 : \text{float}} \text{ DIVFLOAT} \qquad \frac{\Gamma \vdash e_1 : \text{bool} \quad \Gamma \vdash e_2 : \text{bool}}{\Gamma \vdash e_1 \&\& e_2 : \text{bool}} \text{ AND} \\
\\
\frac{\Gamma \vdash e_1 : \text{bool} \quad \Gamma \vdash e_2 : \text{bool}}{\Gamma \vdash e_1 || e_2 : \text{bool}} \text{ OR} \qquad \frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 = e_2 : \text{bool}} \text{ EQ} \qquad \frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 <> e_2 : \text{bool}} \text{ NEQ} \\
\\
\frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 < e_2 : \text{bool}} \text{ LT} \qquad \frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 <= e_2 : \text{bool}} \text{ LEQ} \qquad \frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 > e_2 : \text{bool}} \text{ GT} \\
\\
\frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 >= e_2 : \text{bool}} \text{ GEQ}
\end{array}$$

There are quite a few rules here, but in essence, they all say: if e_1 and e_2 are some type in a given context then $e_1 \square e_2$ is some type in the same context.

Example B.3.1. The following is derivation of the fact that $x + 2$ is of type `int` in the context $\{x : \text{int}\}$.

$$\frac{\frac{\{x : \text{int}\} \vdash x : \text{int}}{\text{VAR}} \quad \frac{\{x : \text{int}\} \vdash 2 : \text{int}}{\text{INTLIT}}}{\{x : \text{int}\} \vdash x + 2 : \text{int}} \text{ ADDINT}$$

Semantics

$$\begin{array}{c}
\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v = v_1 + v_2}{e_1 + e_2 \Downarrow v} \text{ADDINTE} \qquad \frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v = v_1 - v_2}{e_1 - e_2 \Downarrow v} \text{SUBINTE} \\
\\
\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v = v_1 \times v_2}{e_1 * e_2 \Downarrow v} \text{MULINTE} \qquad \frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_2 \neq 0 \quad v = v_1 / v_2}{e_1 / e_2 \Downarrow v} \text{DIVINTE} \\
\\
\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_2 \neq 0 \quad v = v_1 \text{ (mod } v_2)}{e_1 \text{ mod } e_2 \Downarrow v} \text{MODINTE} \\
\\
\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v = v_1 + v_2}{e_1 + e_2 \Downarrow v} \text{ADDFLOATE} \qquad \frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v = v_1 - v_2}{e_1 - e_2 \Downarrow v} \text{SUBFLOATE} \\
\\
\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v = v_1 \times v_2}{e_1 * e_2 \Downarrow v} \text{MULFLOATE} \qquad \frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v = v_1 / v_2}{e_1 / e_2 \Downarrow v} \text{DIVFLOATE} \\
\\
\frac{e_1 \Downarrow \perp}{e_1 \&\& e_2 \Downarrow \perp} \text{ANDFALSE} \qquad \frac{e_1 \Downarrow \top \quad e_2 \Downarrow v}{e_1 \&\& e_2 \Downarrow v} \text{ANDTRUE} \qquad \frac{e_1 \Downarrow \top}{e_1 \mid\mid e_2 \Downarrow \top} \text{ORTRUE} \\
\\
\frac{e_1 \Downarrow \perp \quad e_2 \Downarrow v}{e_1 \mid\mid e_2 \Downarrow v} \text{ORFALSE} \qquad \frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 = v_2}{e_1 = e_2 \Downarrow \top} \text{EQTRUE} \\
\\
\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 \neq v_2}{e_1 = e_2 \Downarrow \perp} \text{EQFALSE} \qquad \frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 \neq v_2}{e_1 <> e_2 \Downarrow \top} \text{NEQTRUE} \\
\\
\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 = v_2}{e_1 <> e_2 \Downarrow \perp} \text{EQFALSE} \qquad \frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 < v_2}{e_1 < e_2 \Downarrow \top} \text{LTTRUE} \\
\\
\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 \geq v_2}{e_1 < e_2 \Downarrow \perp} \text{LTFALSE} \qquad \frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 \leq v_2}{e_1 <= e_2 \Downarrow \top} \text{LTETRUE} \\
\\
\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 > v_2}{e_1 <= e_2 \Downarrow \perp} \text{LTFALSE} \qquad \frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 > v_2}{e_1 > e_2 \Downarrow \top} \text{GTTRUE} \\
\\
\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 \leq v_2}{e_1 > e_2 \Downarrow \perp} \text{GTFALSE} \qquad \frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 \geq v_2}{e_1 >= e_2 \Downarrow \top} \text{GEQTRUE} \\
\\
\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad v_1 < v_2}{e_1 >= e_2 \Downarrow \perp} \text{GEQFALSE}
\end{array}$$

Again, there are a lot of rules here, but they all in essence say that if e_1 and e_2 evaluate to particular values, then $e_1 \square e_2$ evaluates to the expected value. One case of note is Boolean operators. Note how the rules enforce *short-circuiting*.

Example B.3.2. The following is derivation of the fact that $2 + 3$ evaluates to 5.

$$\frac{\frac{2 \Downarrow 2}{\text{INTLIT}} \quad \frac{2 \Downarrow 3}{\text{INTLIT}}}{2 + 3 \Downarrow 5} \text{ADDINT}$$

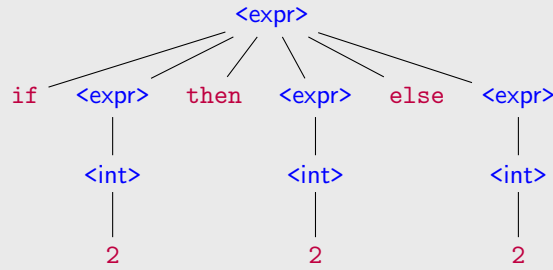
B.4 If-Expressions

Syntax

$\langle \text{expr} \rangle ::= \text{if } \langle \text{expr} \rangle \text{ then } \langle \text{expr} \rangle \text{ else } \langle \text{expr} \rangle$

If e_1 is a well-formed expression and e_2 is a well-formed expression and e_3 is a well-formed expression, then $\text{if } e_1 \text{ then } e_2 \text{ else } e_3$ is a well-formed expression.

Example B.4.1. Since 2 is a well-formed expression, then $\text{if } 2 \text{ then } 2 \text{ else } 2$ is a well-formed expression, i.e., the following is a valid parse tree.



Typing

$$\frac{\Gamma \vdash e_1 : \text{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau} \text{IF}$$

IF says if e_1 is of type **bool** in the context Γ and e_2 is of type τ in the context Γ and e_3 is *also* of type τ in the context Γ , then $\text{if } e_1 \text{ then } e_2 \text{ else } e_3$ is of type τ in the context Γ .

Example B.4.2. The following is a valid derivation of the fact that $\text{if false then } 2 \text{ else } 3 + 4$ is of type **int** in the empty context.

$$\frac{\frac{\frac{}{\emptyset \vdash \text{false} : \text{bool}} \text{TRUE} \quad \frac{}{\emptyset \vdash 2 : \text{int}} \text{INTLIT}}{\emptyset \vdash \text{if false then } 2 \text{ else } 3 + 4 : \text{int}} \text{IF} \quad \frac{\frac{}{\emptyset \vdash 3 : \text{int}} \text{INTLIT} \quad \frac{}{\emptyset \vdash 4 : \text{int}} \text{INTLIT}}{\emptyset \vdash 3 + 4 : \text{int}} \text{ADDINT}$$

Semantics

$$\frac{e_1 \Downarrow \top \quad e_2 \Downarrow v}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow v} \text{IFTRUE} \quad \frac{e_1 \Downarrow \perp \quad e_3 \Downarrow v}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow v} \text{IFFALSE}$$

▷ IFTRUE says if e_1 evaluates to **true** and e_2 evaluates to v , then $\text{if } e_1 \text{ then } e_2 \text{ else } e_3$ evaluates to v

▷ IFFALSE says if e_1 evaluates to **false** and e_3 evaluates to v , then $\text{if } e_1 \text{ then } e_2 \text{ else } e_3$ evaluates to v

In either case, we do not evaluate the irrelevant branch.

Example B.4.3. The following is a valid derivation of the fact that `if false then 2 else 3 + 4` evaluates to 7.

$$\frac{\frac{\text{false} \Downarrow \perp}{\text{false} \Downarrow \perp} \text{FALSEE} \quad \frac{\frac{\overline{3 \Downarrow 3}}{\text{INTLITE}} \quad \frac{\overline{4 \Downarrow 4}}{\text{INTLITE}}}{3 + 4 \Downarrow 7} \text{ADDITE}}{\text{if false then 2 else } 3 + 4 \Downarrow 7} \text{IFFALSE}$$

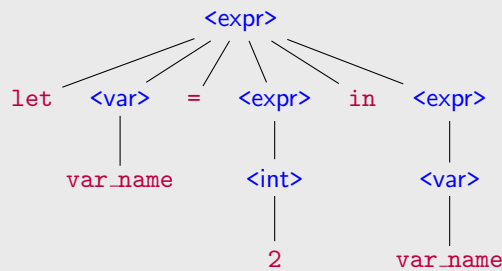
B.5 Let-Expressions

Syntax

```
<expr> ::= let <var> = <expr> in <expr>
```

If x is a variable and e_1 is a well-formed expression and e_2 is a well-formed expression, then **let** $x = e_1$ **in** e_2 is a well-formed expression.

Example B.5.1. Since `var_name` is a well-formed variable name and `2` is a well-formed expression then `let var_name = 2 in var_name` is a well-formed expression, i.e., the following is a valid parse tree.



Typing

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau_2} \text{LET}$$

LET says if e_1 is of type τ_1 in the context Γ , and e_2 is of type τ_2 in the context Γ *with the variable declaration* $(x : \tau_1)$ *added to it*, then **let** $x = e_1$ **in** e_2 is of type τ_2 in the context Γ .

Example B.5.2. The following is a valid derivation.

$$\frac{\frac{}{\{\} \vdash 2 : \text{int}} \text{INTLIT} \quad \frac{\frac{}{\{\text{y} : \text{int}\} \vdash \text{y} : \text{int}} \text{VAR} \quad \frac{}{\{\text{y} : \text{int}\} \vdash \text{y} : \text{int}} \text{VAR}}{\{\text{y} : \text{int}\} \vdash \text{y} + \text{y} : \text{int}} \text{INTADD} \quad \frac{}{\{\} \vdash \text{let } \text{y} = 2 \text{ in } \text{y} + \text{y} : \text{int}} \text{LET}$$

Semantics

$$\frac{e_1 \Downarrow v_1 \quad [v_1/x]e_2 = e' \quad e' \Downarrow v}{\text{let } x = e_1 \text{ in } e_2 \Downarrow v} \text{LETE}$$

LETE says if e_1 evaluates to v_1 and e_2 with v_1 substituted for x evaluates to v , then **let** $x = e_1$ **in** e_2 evaluates to v .

Example B.5.3. The following is a valid derivation.

$$\frac{\frac{2 \Downarrow 2}{\text{INTLITE}} \quad \frac{\frac{2 \Downarrow 2}{\text{INTLITE}} \quad \frac{2 \Downarrow 2}{\text{INTLITE}}}{2 + 2 \Downarrow 4} \text{INTADDE}}{\text{let } y = 2 \text{ in } y + y \Downarrow 4} \text{LETE}$$

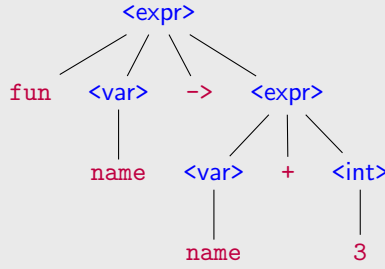
B.6 Functions

Syntax

$$\langle \text{expr} \rangle ::= \text{fun } \langle \text{var} \rangle \rightarrow \langle \text{expr} \rangle$$

If x is a well-formed variable and e is a well-formed expression, then **fun** $x \rightarrow e$ is a well-formed expression.

Example B.6.1. The following is a valid parse tree for the well-formed expression **fun** *name* $\rightarrow$ *name* + 2.



Typing

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{fun } x \rightarrow e : \tau_1 \rightarrow \tau_2} \text{FUN}$$

FUN says if e is of type τ_2 in the context Γ together with the variable declaration $(x : \tau_1)$ added to it, then **fun** $x \rightarrow e$ is of type $\tau_1 \rightarrow \tau_2$.

Example B.6.2. The following is a valid derivation.

$$\frac{\frac{\{z : \text{float}\} \vdash 2 : \text{int}}{\text{INTLIT}}}{\{\} \vdash \text{fun } z \rightarrow 2 : \text{float} \rightarrow \text{int}} \text{FUN}$$

Semantics

$$\frac{}{\text{fun } x \rightarrow e \Downarrow \lambda x.e} \text{FUNE}$$

FUNE says a function **fun** $x \rightarrow e$ evaluates to a special function value that essentially hold the variables x and the body e with the notation $\lambda x.e$.

Example B.6.3. The following is a valid derivation.

$$\frac{}{\text{fun } x \rightarrow x + x \Downarrow \lambda x. x + x} \text{FUN EVAL}$$

B.7 Application

Syntax

$$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle \langle \text{expr} \rangle$$

If e_1 is a well-formed expression and e_2 is a well-formed expression, then $e_1 e_2$ is a well-formed expression.

Typing

$$\frac{\Gamma \vdash e_1 : \tau_2 \rightarrow \tau \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1 e_2 : \tau} \text{APP}$$

APP says if e_1 is of type $\tau_2 \rightarrow \tau$ and e_2 is of type τ_2 , then $e_1 e_2$ is of type τ .

Example B.7.1. The following is a valid derivation.

$$\frac{\frac{\frac{}{\{z : \text{float}\} \vdash 2 : \text{int}} \text{INTLIT}}{\{\} \vdash \text{fun } z \rightarrow 2 : \text{float} \rightarrow \text{int}} \text{FUN} \quad \frac{}{\{\} \vdash 2.0 : \text{float}} \text{FLOATLIT}}{\{\} \vdash (\text{fun } z \rightarrow 2) 2.0 : \text{int}} \text{APP}$$

Semantics

$$\frac{e_1 \Downarrow \lambda x. e \quad e_2 \Downarrow v_2 \quad [v_2/x]e = e' \quad e' \Downarrow v}{e_1 e_2 \Downarrow v} \text{APPE}$$

APPE says if e_1 evaluates to the function value $\lambda x. e$ and e_2 evaluates to v_2 and e with v_2 substituted for x evaluates to v , then $e_1 e_2$ evaluates to v .

Example B.7.2. The following is a valid derivation.

$$\frac{\frac{}{\text{fun } x \rightarrow x + x \Downarrow \lambda x. x + x} \text{FUN EVAL} \quad \frac{2 \Downarrow 2}{2 \Downarrow 2} \text{INTLITE} \quad \frac{\frac{2 \Downarrow 2}{2 \Downarrow 2} \text{INTLITE} \quad \frac{2 \Downarrow 2}{2 \Downarrow 2} \text{INTLITE}}{2 + 2 \Downarrow 4} \text{ADDITE}}{(\text{fun } x \rightarrow x + x) 2 \Downarrow 4} \text{APPE}$$

Remark B.7.1. (IMPORTANT) There is a subtlety to substitution: when a value is substituted for a variable in an expression, *it is converted back into an expression*, e.g., $[2/x](x + x) = 2 + 2$. This will be made more clear when we see how this manifests in our implementation of interpreters. For now, it is important to note that it is simple to convert the values we care about back into expressions via the following mapping:

$integer \mapsto$	$integer\ literal$	(e.g., $2 \mapsto 2$)
$float \mapsto$	$float\ literal$	(e.g., $3.5 \mapsto 3.5$)
$\top \mapsto$	$true$	
$\perp \mapsto$	$false$	
$\lambda x.e \mapsto$	$fun\ x\ -> e$	

Bibliography

- [1] V Aho Alfred, S Lam Monica, and D Ullman Jeffrey. Compilers: Principles, Techniques & Tools. Pearson Education, 2007.
- [2] Gerhard Gentzen. Investigations into Logical Deduction. American philosophical quarterly, 1(4):288–306, 1964.
- [3] Robert Harper. Practical Foundations for Programming Languages. Cambridge University Press, 2016.
- [4] Benjamin C Pierce. Types and Programming Languages. MIT press, 2002.
- [5] Norman Ramsey. Programming Languages: Build, Prove, and Compare. Cambridge University Press, 2022.
- [6] Michael Scott. Programming language pragmatics. Morgan Kaufmann, 2000.